ENEC: A Lossless AI Model Compression Method Enabling Fast Inference on Ascend NPUs

Jinwu Yang*†, Jiaan Wu†, Zedong Liu*†, Xinyang Ma*†, Hairui Zhao*†, Yida Gu*†, Yuanhong Huang*†, Xingchen Liu*†, Wenjing Huang*†, Zheng Wei*, Jing Xing*, Yili Ma*, Qingyi Zhang‡, Baoyi An‡, Zhongzhe Hu‡, Shaoteng Liu‡, Xia Zhu‡, Jiaxun Lu‡, Guangming Tan*†, and Dingwen Tao*†

*Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China

†University of Chinese Academy of Sciences, Beijing, China

‡Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

***Abstract*—The rapid scaling of Large Language Models presents significant challenges for their deployment and inference, particularly on resource-constrained specialized AI hardware accelerators such as Huawei's Ascend NPUs, where weight transfer has become a critical performance bottleneck. While lossless compression can preserve model accuracy and reduce data volume, existing lossless compression algorithms exhibit extremely low throughput when ported to the Ascend NPUs. In this paper, we propose ENEC, a novel lossless compression method specifically customized for AI model weights and optimized for Ascend Neural Processing Units. ENEC adopts a block-based fixed-length encoding scheme and incorporates a series of optimizations: bit-width quantization with hierarchical halving bit-packing, vectorized branch-free integer transformation, and dependency-decoupled intra-segment scan. Experimental results demonstrate that ENEC outperforms existing state-of-the-art NPU compressors in both compression ratio and throughput. Compared to leading GPU solutions, ENEC achieves a 3.43× higher throughput than DietGPU and a 1.12× better compression ratio than nvCOMP. By reducing weight transmission overhead, ENEC significantly improves end-to-end inference performance, achieving up to a 6.3× speedup. On Ascend NPUs, ENEC is the first open-source lossless compression algorithm for model weights that achieves performance comparable to state-of-the-art GPU compressors, offering an effective solution for deploying large-scale AI models.**

***Index Terms*—Lossless Compression, Large Language Models, Ascend NPU, Model Weights.**

I. Introduction

Large Language Models (LLMs) have demonstrated remarkable performance across diverse application domains [1], [2], [17], driving significant demand for efficient training and inference. In response, a range of specialized AI accelerators has been developed, including the Cerebras CS2 [8], SambaNova SN30 [49], and the Groq GroqChip [22]. Among these, Huawei's Ascend Neural Processing Units (NPUs) stand out due to their high computational throughput, strong energy efficiency, and favorable cost–performance characteristics compared to conventional GPUs. Notably, systems such as Huawei's CloudMatrix384 platform, equipped with CloudMatrix-Infer [66], demonstrate impressive scalability on large-scale AI workloads.

Dingwen Tao is the corresponding author (taodingwen@ict.ac.cn).

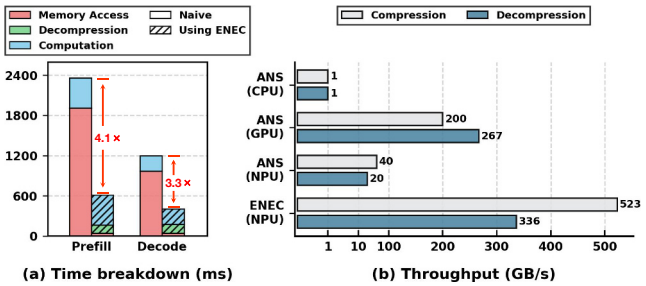

Fig. 1: Time breakdown of Qwen3-32B Inference on NPU 910B2 (left) and throughput of ANS across multiple platforms (right).

Despite the hardware advances, the rapid growth in model size poses severe challenges for deployment and inference efficiency [42], [59], [63], especially in resource-constrained environments. For instance, the Llama-3.1-405B model [17] contains 405 billion parameters and requires approximately 910 GB of memory for full inference in BFloat16 precision [12], [15], [17], [29], [50], exceeding the total HBM capacity of even high-end NPU servers equipped with 8 NPU 910B2s (each with 64 GB HBM). Consequently, deploying such massive models requires distributing model weights across CPUs and NPUs. The frequent CPU-NPU data transfers for accessing remote weights during computation have thus become a critical performance bottleneck [28], [30], [37]. As Figure 1(a) illustrates, memory access constitutes the dominant factor in inference latency for the Qwen3-32B model, accounting for 78% to 85% of the total execution time in both the prefill and decode stages [50]. This severe I/O bottleneck effectively marginalizes computation, drastically underutilizing the hardware. Consequently, this naive deployment strategy results in latencies that are 4.1× and 3.3× higher than our optimized ENEC approach for the prefill and decode stages, respectively. Such overhead not only intensifies the performance bottleneck but also escalates operational costs and hinders accessibility, ultimately limiting the practical deployment of state-of-the-art (SOTA) LLMs.

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00100

To address the challenges associated with model scaling, numerous weight compression techniques have been proposed, including quantization [20], [41], [55], pruning [4], [19], and low-rank decomposition [51], [58]. These methods substantially reduce memory footprint and computational overhead, enabling faster inference under resource constraints. However, they are inherently lossy and often alter the model's output distribution, potentially compromising accuracy and reliability. In contrast, lossless compression techniques—such as entropy coding based on Asymmetric Numeral Systems (ANS) [18], [40], [47], [52], [56]—preserve full model fidelity by ensuring exact reconstruction of the original weights. Notable examples include general-purpose algorithms like Zstandard (Zstd) [10] and nvCOMP [45], as well as specialized encoders tailored for model weights, such as ZipNN [24], DFloat11 [62], and Huff-LLM [60], all of which have demonstrated strong performance on conventional CPU or GPU platforms.

Lossless compression for LLMs is particularly needed for Ascend NPUs because these accelerators—specifically engineered for AI models—are typically highly capable in terms of computational resources, such as matrix and tensor cores, while their HBM capacity and NPU–CPU bandwidth remain limited. Thus, effective lossless compression on NPUs can help bridge this gap. However, an open-source lossless compressor for model weight compression on NPUs is notably absent. To investigate this opportunity, we ported an ANS compressor—an algorithm known for combining the speed of Huffman coding [25]–[27], [43], [48], [57] with compression efficiency approaching that of arithmetic coding [34]—from conventional CPU and GPU environments to the NPU. Unfortunately, our implementation achieved *exceptionally poor throughput*, as shown in Figure 1(b). This performance limitation is particularly critical in inference scenarios, where compressed weights must be decompressed in real time, making the decompression phase itself a severe system bottleneck. Moreover, the issue is not unique to ANS; similar throughput degradation was observed when porting other lossless algorithms such as LZ77 [65]. We conclude that existing lossless compressors are fundamentally incompatible with Ascend architecture. The SIMD-based vectorized design lacks conditional branching, scatter/gather capabilities, and efficient variable-length memory handling—severely limiting parallelization and throughput of traditional compression algorithms, as further analyzed in Section IV-A.

This paper proposes Efficient NPU-Enhanced Compression (ENEC), a novel hardware-software co-designed algorithm for model weights. Designed to harness the full capabilities of Ascend NPUs, ENEC uses a block-based, fixed-length approach with NPU-specific optimizations like bit-width quantization with hierarchical halving bit-packing that boosts compression speed without sacrificing the compression ratio; a table-based mapping strategy combined with a branch-free integer transform, which replaces memory accesses with arithmetic computations to optimize the statistical characteristics of model weights; and an intra-segment dependency decoupled scan for efficient prefix sum. ENEC attains a compression throughput ranging from 263 to 523 GB/s and a decompression throughput of 188 to 336 GB/s, outperforming state-of-the-art NPU compressors by up to 2.47× and 2.11× while maintaining high compression ratio. This breakthrough substantially reduces data transmission overhead, enabling up to a 6.3× speedup in end-to-end inference.

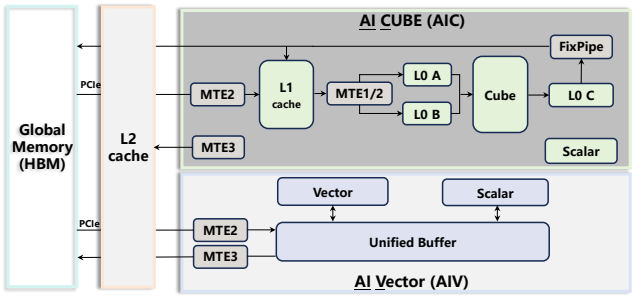

Fig. 2: DaVinci architecture (decoupled AIC and AIV).

Our contributions are summarized as follows:

- We obtain several key observations through in-depth analyses of various AI model weights, which informed our basic design and optimizations.
- We design an lossless encoding method for Ascend NPUs that overcomes architectural constraints, based on our comprehensive analysis of lossless compression incompatibility and an extension of the LC framework.
- We propose a set of optimizations to achieve both high compression ratio and high throughput on Ascend NPUs, including bit-width quantization with hierarchical halving bit-packing, vectorized branch-free integer transformation, and intra-segment dependency–decoupled scan.
- Evaluated on Ascend 910B2 with 10 real-world AI models, ENEC outperforms SOTA NPU compressors in compression ratio and throughput. Compared to GPU compressors, ENEC achieves 3.43× higher throughput than DietGPU and a 1.12× better ratio than nvCOMP, yielding up to a 6.3× end-to-end inference speedup.

To the best of our knowledge, ENEC[1] is the first open-source compressor for AI model on Ascend NPUs that achieves performance comparable to SOTA GPU compressors while maintaining the model weight compression ratio.

II. Background

A. Ascend NPUs

The Ascend NPUs are specialized accelerator chips built on the DaVinci architecture [39] for deep learning workloads.

Compute Architecture and Core Specialization. The Ascend NPUs consist primarily of memory units and computing units. The main computing components are AI Cores.

[1]Our code is available at https://github.com/hpdps-group/ENEC.

One AI core typically integrates one AI Cube (AIC) unit and two AI Vector (AIV) units. For instance, the Ascend 910B2 NPU incorporates 24 such AI Cores, providing a total of 24 AIC units and 48 AIV units. The AIC is optimized for high-performance matrix operations, offering exceptional throughput for dense linear algebra, albeit with limited operational flexibility. In contrast, the AIV supports large-scale vectorized operations, including data gathering, reduction, and other collective operations.

Memory Hierarchy and Data Path. On the memory side, the Ascend NPUs feature a high-bandwidth memory (HBM) and an L2 cache shared across all AI Cores. For AIV operations, input data must first be loaded into a local buffer known as the Unified Buffer (UB), typically on the order of kilobytes. The AI Cube unit employs multiple levels of on-chip buffers such as L1, L0A, L0B, L0C, and FP buffers, as illustrated in Figure 2. Data movement between different memory hierarchies is managed by Memory Transfer Engines (MTEs), which not only handle data transfers but can also perform in-flight data format and type conversions.

Programming Model and Execution Pipeline. Huawei introduced AscendC, a C++ programming model for Neural Networks (CANN) [14], to enable high-performance operator development on Ascend NPUs. AscendC employs a multi-pipeline abstraction to enable fine-grained hardware control, allowing developers to fully exploit hardware's potential. It simplifies programming through key abstractions like tensors and queues. A tensor encapsulates data that will be operated on by the AIC/AIV units, while queues handle synchronization: after an operation completes, the result is enqueued (EnQue), and dependent operations dequeue (DeQue) it. More details can be found in the user documentation [14].

B. Lossless Floating-Point Compressors

General-purpose Lossless Floating-point Compressors. Lossless compressors for floating-point data enable compression without information loss, making it valuable for applications like image [16], time series data [36], [38], and scientific data [5], [9], [33]. The core techniques of lossless compression primarily fall into two categories: one leverages the identification of spatial redundancy patterns, while the other exploits the varying frequencies of different symbols. The first category is characterized by LZ-based compression methods (e.g., LZW [53], LZ78 [64], LZ77 [65]) and related optimizations [61], which operate by sequentially scanning the data stream and replacing repeated substrings with shorter references. The second category is represented by entropy coding techniques such as Huffman coding [25]–[27], [43], [48], [57] and arithmetic coding [34], which assign shorter bit representations to more frequently occurring symbols, thereby reducing the overall bit length.

Dedicated Lossless Floating-point Compressors for AI Model. Despite the high compression efficiency and throughput of general-purpose compressors like Zstd [10] on typical data types, their performance often degrades significantly on model weight data. Recently, several methods (e.g., ZipNN [24] and its extension [23], DFloat11 [62], Huff-LLM [60]) have identified that the high randomness of floating-point mantissas interferes with the compressibility of exponents, leading to suboptimal results. Moreover, DietGPU [31], the ANS-based [18], [40], [47], [52], [56] compressor on GPU, has introduced a dedicated float codec specifically designed for model floating-point data. To address this, they propose separating the exponent and mantissa components before compression. While this approach improves compression ratios, the exponent compression still relies on variable-length coding, which is poorly suited for efficient implementation on Ascend NPUs due to irregular memory access and control flow, thereby limiting their practical deployment efficiency on Ascend NPUs. Recently, Huawei has developed a compression algorithm named HANS [3]; however, its compression ratio and throughput performance still remain limited and it is also closed-source.

C. LC Framework to Search Optimal Compression Algorithm

To achieve efficient compression on Ascend NPUs, lightweight operations are essential. For this purpose, we have enhanced the emerging and significant open-source data compression tool—the LC framework [7] and utilized it to search for lightweight component combinations that deliver either the optimal compression ratio or the highest throughput. This framework provides a wide range of common compression components and preprocessing methods, among which the Reducer is the sole component used for shortening data sequences, employing techniques such as HCLOG, RLE, RRE, and RZE to leverage various types of data redundancy.

Among these techniques, HCLOG employs a grouped bit-packing strategy: it divides a 16 KB data block into a fixed number of 32 sub-chunks, calculates the minimum leading zero count in each sub-chunk as metadata, and stores only the valid bits. However, a single outlier can force the entire sub-chunk to adopt a higher bitwidth, thereby reducing the compression ratio. To address this issue, we have extended the LC framework with a series of HCLOG compressors that support varying numbers of sub-chunks.

III. Data Analysis of AI Model Weights

Model weights are typically stored in one of three floating-point formats: FP32, FP16, or BF16. Recently, BF16 has gained popularity for large models due to its wider exponent range, which offers training stability [32], [35], [54]. Although lower-precision integer formats such as INT8 [13] and FP8 [44] have been explored for model compression, they are not the primary focus of this study for several reasons. First, INT8 and FP8 offer lower precision than BF16, risking accuracy. Second, INT8 and FP8 formats typically require quantization, which constitutes a separate lossy compression step and offers limited additional compressibility

beyond the quantization itself. Third, while the Ascend NPU's Cube units provide hardware support for INT8, the AIV API—the core framework of our implementation—lacks the necessary programming interfaces and instruction sets for the bit-manipulation and transformation tasks required by our method. Moreover, FP8 is entirely unsupported by the hardware, further precluding its use. Since models in other floating-point formats exhibit similar statistical properties, our analyses focus on BF16 as a representative case.

TABLE I: LC search results. "Others" represents the set of all non-HCLOG algorithmic variants within the LC framework.

LC-Alg	Models		
	deepseek-llm-7b-base [12]	Qwen3-8b [50]	Llama-3.1-8B-Instruct [17]
HCLOG	**98.23%**	**98.30%**	**99.36%**
Others	1.77%	1.70%	0.64%

Observation 1: The exponent is more compressible than the sign and mantissa. BF16 format uses 1 sign bit, 8 exponent bits, and 7 mantissa bits. Analysis of DeepSeek weights shows sign and mantissa bits have uniform distribution, while exponents are highly non-uniform. Entropy calculations confirm: sign and mantissa have high entropy (7.97 bits), making compression difficult; exponents show low 2.58-bit entropy with strong compression potential. This insight aligns with findings from recent studies such as ZipNN [24] and DFloat11 [62].

Observation 2: In the LC framework, the HCLOG variant achieves the highest compression ratio in most cases. We use model weight tensors as input. Specifically, we partition the large-scale model parameters into multiple smaller files to enable a more granular analysis. For each file, we independently invoke the improved LC framework [7] to search for its optimal configuration, thereby identifying a lightweight compressor that maximizes compression ratios. As shown in Table I, results across various model weight datasets consistently show the HCLOG variant achieves the highest ratio in most cases.

Observation 3: Exponent values are highly constrained, concentrated within a narrow continuous range. Our further analysis reveals that the actual range of exponent values is highly constrained, with many potential values completely absent from the weights. Comprehensive examination demonstrates that these exponent values are concentrated within a narrow continuous range.

Observation 4: The bit widths of the data grouped by the HCLOG grouping method after mapping by frequency sorting are obviously consistent. By applying frequency-based mapping and using the grouping method from the HCLOG approach mentioned in Section II-C, we observe that most grouped data has a bit width $\leq$ m, while a small portion has a bit width $>$ m.

Observation 5: A significant linear relationship exists between the weight exponent values and their frequency rankings. We conduct an in-depth analysis of the frequency distribution of weight exponent values and observed a distinct negative linear relationship between the exponent values and their frequency ranking. As shown in Figure 3, this relationship can be effectively fitted by a linear function such as $Y = -1.00X + 123.00$, where the X-axis represents the exponent value and the Y-axis represents the corresponding frequency ranking (a lower ranking indicates higher frequency). This highly regular and predictable distribution characteristic serves as a critical foundation for efficient compression of model weights. It is worth noting that a small number of high exponent value points marked by the red elliptical region in the figure deviate from the main linear trend, representing rare outliers.

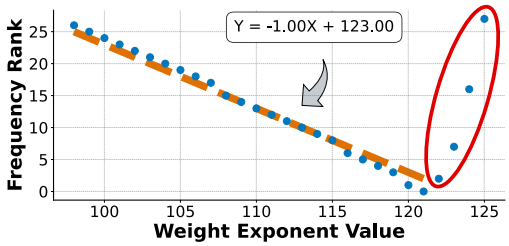

Fig. 3: Linear relationship between exponent values and frequency rankings in model weights. The red circle indicates outliers.

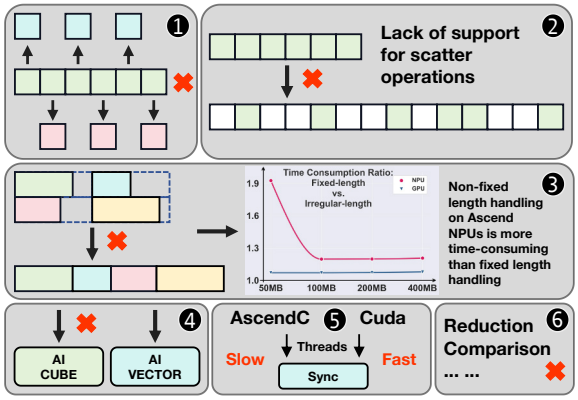

Fig. 4: Execution challenges and constraints on Ascend NPUs.

IV. Architecture Analysis and Basic Design

A. Architecture Analysis of Lossless Compression Incompatibility on Ascend NPUs

As Figure 4 shows, the fundamental incompatibility between Ascend NPUs and lossless compression algorithms stems from architectural mismatches at multiple levels:

❶ The current Ascend architecture, based on SIMD, does not support flexible element-wise operations such as conditional if branching. Furthermore, it has limited instructions for integer arithmetic, making it particularly unsuitable for entropy coding.

❷ The lack of instructions for converting between contiguous and non-contiguous memory layouts, such as `scatter`, severely degrades decoding performance.

❸ Handling and transferring variable-length memory operations is inefficient and time-consuming, severely impacting algorithms like Huffman coding.

❹ Compression computations are primarily vector-based element-wise operations and cannot directly utilize the Ascend's Cube unit designed for matrix operations. Therefore, we construct a collaborative inference pipeline where compression is handled by the AIV and model operations are executed by the AIC.

❺ AscendC lacks fast inter-thread synchronization comparable to CUDA, as Ascend NPUs treat each AI Core as a single, heavyweight thread. The architecture relies on a pipelined dataflow model, optimizing task-queue-driven overlaps between data movement and computation within a thread, rather than managing fine-grained concurrency across massive threads.

❻ It is challenging to implement lossless compression algorithms that require extensive non-arithmetic operations (such as comparisons and reductions), including Run-Length Encoding (RLE), LZ4, LZ77, and other similar dictionary-based methods.

These architectural mismatches explain the fundamental incompatibility between existing lossless compression algorithms and Ascend NPUs. The lack of conditional branching, scatter/gather instructions, and efficient variable-length memory operations renders traditional entropy coding and dictionary-based methods impractical on this platform. Moreover, the absence of lightweight inter-thread synchronization and reliance on a pipelined dataflow model within each AI core restrict parallelization strategies common in GPU-based compressors. These insights motivate our co-design approach: rather than force-fitting existing algorithms, we align compression primitives with Ascend's vectorized, branch-free execution model and memory hierarchy constraints. Building on this understanding, we next present ENEC—a lightweight compression algorithm crafted to navigate these architectural limitations while preserving lossless fidelity.

B. Basic Design of ENEC

Building upon the architectural constraints identified in Section IV-A, we now present the basic design of ENEC—a lightweight compression algorithm, as shown in the pseudocode (Algorithm 1). While the architectural analysis reveals fundamental incompatibilities, it also illuminates the path forward: by aligning our algorithm with the Ascend's inherent strengths and working around its constraints, we can achieve efficient lossless compression.

Guided by Observations 1 and 2 from Section III, our fundamental compression algorithm proceeds as follows. During the compression phase, each thread processes a data block of 8,192 elements at a time. The exponents are separated from the floating-point data (while signs and mantissas are stored directly), and frequency statistics are collected (Line 2). The sorted frequency indices serve as mapping values to

Algorithm 1: Basic Design of ENEC

The right-side markers [B1-B4] represent the primary architectural bottlenecks on Ascend NPUs addressed in Section V

Input : W: Raw Weights (BF16/FP16/FP32), T_{freq}: Frequency Table
Output: $Stream$: Compressed Binary Output

1 **Compression (Processing 8,192 elements per block):**
// Component Separation
2 $\{E, S, M\} \leftarrow$ Split(W)
3 $E' \leftarrow T_{freq}[E]$ // **[B1] Gather Lookup**
4 **(Limit: High latency in non-contiguous memory access, 35% overhead)**
// Group-based $bit\text{-}width$ calculation
5 Divide E' into groups of length L
6 **for** *each group* G **do**
7 $bit\text{-}width \leftarrow \lceil \log_2(\max(G)) \rceil$ // **[B2] Reduction Max**
8 **end**
9 **(Limit: Serial dependency in vector units, 40% overhead)**
// Bit-packing
10 **while** *bits remaining in* E' **do**
// Buffer Accumulation
11 Fill 8,192 lanes in 32KB $buffer$
12 **if** *lane status* $\geq$ *16 bits* **then**
13 Output $lower$ 16 $bits$ and $bit\ mask$ to $Stream$
14 $lanes \leftarrow lanes \gg 16$
15 **end**
16 **end**
17 **Decompression:**
// Offset calculation and bit-unpacking
18 $Mask \leftarrow$ Convert $bit\ mask$ to $\{0, 1\}$ integers
19 $Offsets \leftarrow$ PrefixSum($Mask$) // **[B3] Prefix Sum**
20 **(Limit: Cross-lane data dependency within 32B segments, 30% overhead)**
21 $Lower$ 16 $bits \leftarrow$ Get from $Stream$ using $Offsets$
22 $E' \leftarrow buffer + Lower$ 16 $bits$
23 $E \leftarrow T_{freq}^{-1}[E']$ // **[B4] Gather Lookup**
24 **(Limit: Inefficient vector unit utilization in gather, 45% overhead)**
25 $W \leftarrow$ Combine(E, S, M)
26 **return** W

achieve more compact encoding (Line 3). Within this 8,192-element block, data is further divided into smaller groups of length L, and the bit width required for the maximum value in each group is calculated (Lines 5-8). Each thread utilizes a dedicated 32KB buffer (with 8,192 32-bit lanes corresponding to 8,192 elements), where each element is written into its respective lane. As the number of processed blocks increases, the lower 16 bits of all lanes that have accumulated full 16 bits are gathered to output and a bit mask is generated, followed by a vectorized right-shift operation to prepare for the next round (Lines 10-16). This iterative process continues until all data blocks are fully processed. Finally, the compressed content from each thread's buffer is saved into the compressed file for decoding purposes.

During the decompression phase, bit-unpacking operations are first performed to reconstruct the grouped data. The inverse bit-packing employs a reverse gather operation, where the necessary offsets for the reverse gather are precisely computed using a prefix sum, which is derived by first converting the bit mask into a sequence of 0 and 1 integers (Lines 18-22). Subsequently, by executing a lookup in the frequency mapping table, the values are converted back to their original exponent values (Line 23). By combining these restored exponents with the directly stored sign and mantissa bits, the original floating-point data can be perfectly reconstructed (Line 25).

Key Bottlenecks of Basic Design. However, the basic

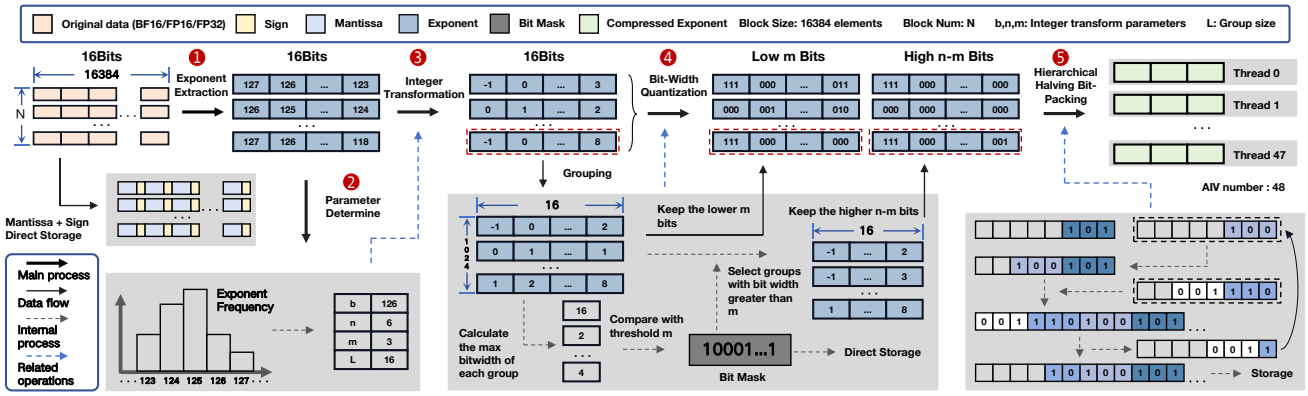

Fig. 5: Overview of optimized ENEC compression design on Ascend NPUs.

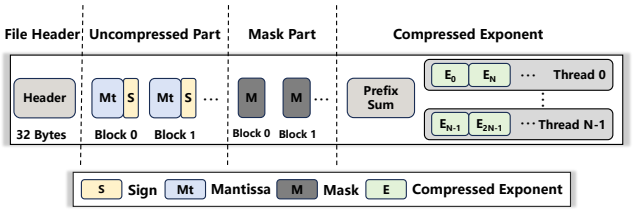

Fig. 6: Compressed stream layout. The bit mask is used to distinguish anomalous groups within each data block. Prefix sums provide direct starting positions for each thread's compressed data.

design does not deliver optimal performance. Performance analysis of the compression kernel reveals that gather table lookups account for 35% of the time (Line 3), while reduction max operations consume 40% (Line 7). Similarly, analysis of the decompression kernel shows that prefix sum operations take 30% of the time (Line 19), with gather table lookups occupying 45% (Line 23). Collectively, these limitations result in low throughput, preventing the system from achieving high-performance efficiency. Therefore, in Section V, we comprehensively optimize the compression and decompression processes of the basic design based on Observations 3, 4 and 5 from Section III.

V. Optimizations of ENEC

Drawing on the observations from Section III and hardware characteristics from Section IV, this section details the optimizations on the basic ENEC from Section IV-B in their order of implementation, directly corresponding to the ablation studies presented in Section VI. Specifically, we first present the optimized ENEC algorithm in Section V-A. Subsequently, from Section V-B to V-E, we delve into the specific optimization methods and fine-grained tuning mechanisms employed to maximize performance.

A. Overview of optimized ENEC

Our optimized ENEC approach, as illustrated in Figure 5, processes model weights by dividing them into fixed-length data blocks that are processed in parallel by multiple threads (with the number of threads determined by the number of AIV cores) in a cyclic manner.

During compression, the process begins by extracting exponent(❶) and calculating the exponent frequency to determine optimal parameters(❷). After applying the branch-free integer transformation(❸) in Section V-C, the data is grouped and the bit-width of each group's maximum value is determined. This bit-width is compared against a threshold to generate and store a compact bit mask and then it distinguishes elements based on this bit mask(❹). Finally using hierarchical halving bit-packing(❺) in Section V-B to generate compressed file (Figure 6).

During decompression, the bit mask is first read and converted to integer values, followed by computing prefix sums using the intra-segment dependency decoupled scan (Section V-D) to obtain offset for the reverse gather operation on the compressed data stream. Subsequently, an inverse vectorized, branch-free integer transformation is performed to restore the exponent data. Finally, the original data is reconstructed by reading the stored sign and mantissa bits.

B. Bit-Width Quantization with Hierarchical Halving Bit-Packing

In the basic design, the variable bit-width packing requires inefficient reduction max and multiplication/division operations on Ascend NPUs. To address this, based on the observation 3 and 4 in Section III, we adopt the lossless bit-width quantization with hierarchical halving bit-packing techniques. The data block is organized using a group-interleaved scheme, where each group comprises L elements. The number of bits required to pack a group is determined by calculating the bit width of the largest element within that group. We propose a two-level bit-width quantization strategy: if the required bit width is less than or equal to a threshold m, all elements within the group are stored uniformly using m bits; otherwise, all elements are stored using n bits, where n represents the minimum number of bits required to represent different occurring exponents. This approach enables substituting the computationally expensive reduction max operation with efficient bitwise OR operation.

To achieve both extreme compression and high performance for tensor blocks under strict hardware alignment

Algorithm 2: Hierarchical Halving Bit-Packing with Byte Normalization (Vectorized)

Input : $data[0..N-1]$: Array of N 16-bit data values ($N = 2^k$);
a: Bit-width of each input value ($0 < a \leq 8$)
Output: $byte\_stream$: Compressed byte-aligned output stream

// Initialization of packing parameters
1 $total\_length \leftarrow 0$
2 $width \leftarrow a$
3 $length \leftarrow N$
4 $normalized\_bytes \leftarrow$ empty vector
5 **while** $width > 0$ **do**
// Hierarchical halving: doubling width by merging pairs
6 **while** $length > 1$ ***and*** $width < 8$ **do**
7 $length \leftarrow length/2$
8 **for** $i \leftarrow 0$ ***to*** $length - 1$ **do**
9 $data[i] \leftarrow data[i] \mid (data[i + length] \ll width)$
10 **end**
11 $width \leftarrow width \times 2$
12 **end**
// Extract aligned bytes from the packed 16-bit lanes
13 **for** $j \leftarrow 0$ ***to*** $length - 1$ **do**
// Mask out the lower 8 bits and update residual data
14 $temp\_bytes[j] \leftarrow (data[j] \ll 8) \gg 8$
15 $data[j] \leftarrow data[j] \gg 8$
16 **end**
17 Append $temp\_bytes$ to $normalized\_bytes$
18 $width \leftarrow width - 8$
19 $total\_length \leftarrow total\_length + length$
20 **end**
// Padding to ensure even length for 16-bit aligned output
21 **if** $total\_length\%2 \neq 0$ **then**
22 Append 0 to $normalized\_bytes$
23 $total\_length \leftarrow total\_length + 1$
24 **end**
// Final stream construction via vectorized concatenation
25 **for** $i \leftarrow 0$ ***to*** $total\_length/2 - 1$ **do**
26 $output\_data \leftarrow normalized\_bytes[i] \mid (normalized\_bytes[i + total\_length/2] \ll 8)$
27 Append $output\_data$ to $byte\_stream$
28 **end**
29 **return** $byte\_stream$

constraints, we introduce an iterative bit-packing algorithm. The core of this method, as shown in the pseudocode (Algorithm 2), lies in lane folding and byte normalization. The algorithm begins with a block of N elements (where N is a power of two), each of 16-bit width but retaining only the least significant a bits. It iterates until the effective bit width of each element is reduced to 8 bits. In each iteration, the block is treated as a sequence of logical lanes. The folding step logically left-shifts each element in the lower half by a bits and performs an element-wise Or operation with the corresponding element in the upper half (Lines 9-11). This merges two a-bit payloads into a single physical storage location, resulting in a new block of $N/2$ elements, each with an effective width of $2a$ bits (Line 12).

When the folding operation causes the effective bit width to exceed the 8-bit byte boundary, *byte normalization* is triggered. As the folding is iterative, this effective bit width grows with each iteration, potentially surpassing 8 bits multiple times. The data is then split into (i) the lower 8 bits, forming a storable byte, and (ii) the remaining bits, which are treated as overflow. All overflow segments are collected into a new sub-block and processed recursively by the same algorithm (Lines 14-21). Then, by appending zero bytes to the end of the byte stream, the total length is aligned upward

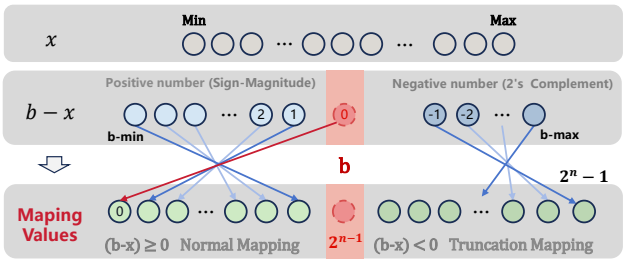

Fig. 7: Vectorized branch-free integer transformation.

to the nearest multiple of 2 (Lines 23-26). The normalized bytes are consolidated in a final folding pass to form the output stream (Lines 28-32). The bit-unpacking process is precisely the inverse of the bit-packing process.

Consequently, as illustrated in Figure 5, the least significant m bits are truncated and compressed via hierarchical halving bit-packing. For groups needing more than m bits, the higher $(n - m)$ bits are collected into a 32 KB buffer. Once full, these bits are compressed by hierarchical halving bit-packing, and the buffer is then cleared. During decompression, an inverse `Gather` operation redistributes the decompressed higher $(n - m)$ bits to their original positions. The original values are reconstructed by performing vectorized Or operation with the decompressed lower m bits.

C. Vectorized Branch-Free Integer Transformation

In the basic design, the exponent mapping relies on the slow gather API, based on the observation 5 in Section III, we adopt the linear mapping function $f(x) = b - x$ as the core transformation scheme to optimize this process. As shown in Figure 7, b is a linear mapping parameter determined by the data distribution characteristics. When the input value is greater than the parameter b, the mapping result is negative. This approach cleverly utilizes the properties of two's complement representation to handle negative values. Due to the use of bit-width quantization and the need to ensure correct decoding, an additional bit is required in the higher bit position to determine whether the value is negative: for positive values, the $(n + 1)$-th bit must be 0, and the mapping range is $[0, 2^n - 1]$; for negative values, the $(n+1)$-th bit must be 1, and the mapping range is $[2^n, 2^{n+1} - 1]$. This approach enables the replacement of irregular memory access operations with efficient computation.

The implementation of this mapping primarily consists of three stages. First, a vector addition unit subtracts the preset parameter b from all input elements simultaneously, shifting the center of the data distribution close to zero. Then, a vector multiplication unit inverts the intermediate results to achieve the mathematical transformation $b - x$. For example, with $b = 123$, an input $x = 125$ is transformed to $123 - 125 = -2$, while an input $x = 122$ becomes $123 - 122 = 1$. This combined operation ensures that high-frequency values in the input data are mapped to smaller output values. Finally, we use shift operations to implement

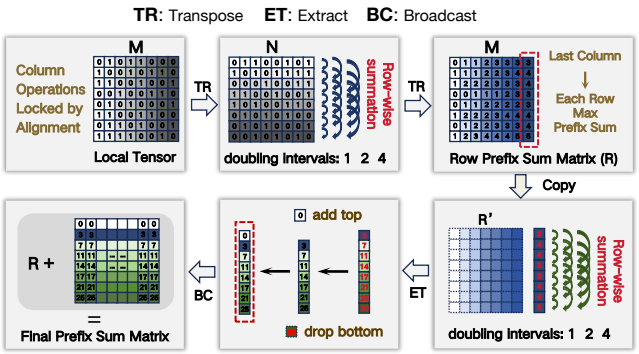

Fig. 8: Prefix sum process. This figure illustrates the step-by-step implementation of the prefix sum algorithm, using an 8×8 binary local tensor M as a working example.

modulo 2^{n+1} and clear higher-bit information, restricting all output values to the predefined range $[0, 2^{n+1} - 1]$. Negative values $-c$ are converted to $2^{n+1} - c$ to achieve a wrapping effect. For instance, in the case where $n = 5$, the previous intermediate result of -2 is converted to $2^6 - 2 = 62$, while the positive result of 1 remains unchanged. This approach ensures computational efficiency while maintaining injectivity, fully leveraging the vectorized computing capabilities of Ascend NPUs and avoiding inefficient branching operations.

D. Intra-Segment Dependency Decoupled Scan

The prefix sum (or scan) is a fundamental parallel primitive. However, its straightforward implementation on modern SIMD architectures, such as Ascend NPUs, is often hindered by stringent hardware constraints. The target operation is to compute the global prefix sum of all elements in an $N \times M$ tensor (where $M = 16$), which is logically treated as a flattened 1D array. The primary bottleneck lies in the Ascend NPUs' memory model, particularly the 32-byte alignment requirement for operands in AscendC. Given that the data type used here is half-precision floating-point (`half`, 2 bytes), each row of M elements in the tensor exactly occupies 32 bytes. Architectural constraints prohibit direct SIMD computation between elements residing within the same 32-byte memory segment. This effectively "locks" the direct intra-row prefix sum computation (e.g., $\text{row}[i] \mathrel{+}= \text{row}[i-1]$), as it requires operations between adjacent elements within a single hardware-indivisible block.

To decouple this forbidden intra-segment dependency, we propose the *Intra-Segment Dependency Decoupled Scan (IDD-Scan)*, a multi-stage algorithm that transforms the computation-bound problem into a series of hardware-friendly operations. The workflow of *IDD-Scan* is detailed below and illustrated in Figure 8.

Stage 1: Intra-Row Scan via Matrix Transposition. *IDD-Scan* first computes the prefix sum for each row by transforming it. The local $N \times M$ tensor is transposed into an $M \times N$ matrix. This distributes each original row's elements across M new rows, converting intra-row dependencies into inter-row ones. A parallel prefix sum is then performed on each of the N columns using vectorized additions in $\log_2(M)$ steps. The matrix is transposed back to its original $N \times M$ orientation, resulting in an intermediate matrix $\mathbf{R}$ where each row contains its correct local prefix sum.

Stage 2: Inter-Row Propagation and Final Update. The intermediate matrix $\mathbf{R}$ contains row-local sums but lacks exclusive cumulative offsets from previous rows. A temporary copy $\mathbf{C}$ of $\mathbf{R}$ is created. A hierarchical scan is performed on $\mathbf{C}$'s rows in $\log_2(N)$ steps: at step k, each row $\mathbf{C}[i]$ adds values from row $\mathbf{C}[i - 2^k]$ element-wise. After completion, the last column of $\mathbf{C}$ holds the inclusive cumulative offsets for all rows. Subsequently, the exclusive cumulative offsets are derived from the inclusive cumulative offsets by removing the bottom element and adding a zero element at the top. This $N \times 1$ vector is broadcast across all columns to form an offset matrix. The final result is obtained by adding offset matrix element-wise to the saved matrix $\mathbf{R}$.

E. Parameter Tuning

This procedure analyzes exponent statistics to derive parameters that minimize the average compressed bit-length under a specialized cost model, following systematic phases:

Phase 1: Statistical Pre-Analysis. The initial phase involves constructing a histogram of the exponents extracted from the source data to obtain a frequency distribution for each unique exponent x. This distribution allows for the calculation of the probability $p(x)$ of each value occurring and the identification of the global minimum (l) and maximum (h) values in the exponents.

Phase 2: Global Search for a Suitable Linear Mapping Parameter (b) and Base Bit-Width (n). The algorithm performs an exhaustive search for a linear mapping parameter b across its feasible integer domain. For each candidate b, the requisite base bit-width n is computed as the minimal number of bits required to span the data range relative to b:

$$n = \max\left(\lfloor \log_2(b - l) \rfloor + 1, \lceil \log_2(h - b) \rceil\right) + 1 \tag{1}$$

For each (b, n) pair, all original exponents x are transformed via a mapping:

$$y = (2^n - x + b) \% 2^n \tag{2}$$

A cost function D, defined as the probability-weighted sum of the transformed values, is evaluated:

$$D = \sum_x p(x) \cdot y \tag{3}$$

The pair (b^*, n^*) that results in the minimum value for D is selected. Using the selected parameters (b^*, n^*), the original exponents are transformed via Equation 2. A statistical analysis of the bit-widths required for the transformed values y is then performed. This yields the cumulative distribution function $p(m)$, representing the probability that a value y can be represented using m or fewer bits.

TABLE II: Compression ratio results on different datasets.

Arch	Compressors	BF16					FP16		FP32		
		Falcon	Qwen3-8B	DeepSeek	Qwen3-32B	Llama	Mistral	Diffusion	OLMo	BERT	Wav2Vec
CPU	ZipNN	1.51	1.50	1.51	1.50	1.51	1.19	1.18	1.20	1.20	1.21
GPU	NV_Zstd	1.28	1.27	1.28	1.27	1.29	1.09	1.08	1.08	1.08	1.08
	NV_Deflate	1.28	1.27	1.28	1.27	1.29	1.09	1.08	1.08	1.08	1.08
	NV_GDeflate	1.27	1.27	1.27	1.27	1.28	1.09	1.08	1.07	1.08	1.08
	NV_ANS	1.25	1.24	1.25	1.25	1.27	1.08	1.04	1.06	1.04	1.03
	NV_Bitcomp	1.33	1.32	1.33	1.32	1.32	1.13	1.07	1.14	1.14	1.15
	Diet_ANS	1.23	1.23	1.23	1.23	1.25	1.06	1.05	1.05	1.05	1.05
	Diet_Float	1.48	1.47	1.48	1.47	1.48	1.17	1.16	1.19	1.19	1.19
NPU	HANS	1.34	1.34	1.35	1.34	1.33	1.09	1.05	1.14	1.13	1.13
	ENEC	**1.36**	**1.36**	**1.37**	**1.35**	**1.36**	**1.12**	**1.09**	**1.15**	**1.15**	**1.15**

Phase 3: Selection of Encoding Threshold (m) and Group Length (L). In the final phase, the appropriate encoding threshold m and group length L are determined. Both L and m are treated as parameters. Since Ascend NPUs require data movement to be 32-byte aligned, we must set L $\geq$ 16. The final parameter pair is selected through a joint search to minimize the expected bit length B_{exp}:

$$(m^*, L^*) = \arg\min_{m,L} \left[B_{\text{exp}} = \frac{1}{L} + n + (m-n) \cdot p(m)^L \right] \quad (4)$$

Under parameters (m, L), expected bits per element B_{exp} comprises amortized group mask overhead $\frac{1}{L}$, base bit-width n, and threshold effect $(m-n)\cdot p(m)^L$. For such groups, each element uses m instead of n bits, saving $n-m$ per element, giving expected saving $(n-m)\cdot p(m)^L$ or equivalently adding $(m-n)\cdot p(m)^L$ to baseline n. Optimal (m^*, L^*) minimize B_{exp} by balancing: larger L reduces $\frac{1}{L}$ but lowers $p(m)^L$.

Upon completion of this process, the algorithm outputs the best set of selected parameters, denoted as the chosen linear mapping parameter b^*, the base bit-width n^*, the encoding threshold m^*, and the group length L^*.

VI. Experimental Evaluation

Platforms. All evaluations of NPU-based methods are performed on the Ascend NPU 910B2, which contains 24 Cube Units and 48 Vector Units (with a vector-to-cube unit ratio of 2:1). The host machine is powered by a HiSilicon Kunpeng-920 CPU running Ubuntu 22.04.5 LTS, with NPU driver version 25.0.rc1.1. Our algorithms are implemented in C++17 using the AscendC framework, with toolkit version 8.2.RC1.alpha002 and the same version for the operator kernel package. To ensure a fair comparison, CPU-based and GPU-based methods are evaluated on a performance-similar NVIDIA A800 GPU platform, which is equipped with Intel Xeon 8358P CPUs and uses CUDA version 12.6.

Datasets. We conduct tests on the weights of ten popular open-source AI models, including three FP32 models, two FP16 models, and five BF16 models. All model weights are downloaded directly from the HuggingFace platform. It should be noted that current mainstream large models, such as Qwen and Llama, adopt the BF16 format; therefore, BF16 is our primary focus, as analyzed in Section III. Detailed information about the datasets can be found in Table III.

TABLE III: Popular models' weights data used in this work.

Type	Models	Size (GB)	Layer Size (MB)
BF16	Falcon-7B [46]	14.40	39.38∼563.56
	Qwen3-8B [50]	16.38	8.00∼1187.00
	deepseek-llm-7b-base [12]	13.82	32.00∼800.00
	Qwen3-32B [50]	65.60	10.00∼1483.75
	Llama-3.1-8B-Instruct [17]	16.06	8.00∼1002.00
FP16	CapybaraHermes-2.5-Mistral-7B [29]	14.50	8.00∼250.02
	stable-video-diffusion-img2vid-fp16 [6]	4.27	3.10×10^{-5}∼96.50
FP32	OLMo-1B-hf [21]	5.10	16.00∼393.00
	bert-base-uncased [15]	0.40	0.01∼89.42
	wav2vec2-large-xlsr-53-english [11]	1.20	4.88×10^{-4}∼32.00

Compression Baselines. We compare our approach against SOTA lossless compressors tailored for different hardware platforms: ZipNN [24] for CPU, DietGPU [31] and nvCOMP [45] for GPU, and HANS [3] for Ascend NPUs. Since both DietGPU and nvCOMP include multiple compression algorithms, we use "Diet" to denote DietGPU and "NV" to denote nvCOMP (e.g., Diet_ANS refers to the ANS algorithm in DietGPU). Our primary evaluation metrics are compression ratio, compression throughput, and decompression throughput. ZipNN and Diet_Float employ tail exponent separation on model data, which serves as a key benchmark for achieving high compression ratios. In contrast, other methods highlight the efficiency advantages of our ENEC.

Compression Configurations. For the ENEC method, we perform offline automatic tuning using the search algorithm described in Section V-E, and adopt the resulting configurations (as shown in Table IV) as the actual parameters for online compression in subsequent models. For all other methods, we use their default configurations. Specifically, for baseline methods such as nvCOMP and DietGPU, we utilize their default settings, which are designed to reflect the optimal trade-off between throughput and compression ratio. Although we also performed parameter tuning for these baselines, the resulting performance showed negligible difference. ENEC runs exclusively on the AIV units, which have significantly lower peak compute capacity than a full A800 GPU. Nevertheless, its competitive performance demonstrates the remarkable effectiveness of architectural specialization and justifies our throughput comparisons, highlighting the success of our hardware/software co-design for this dedicated task.

End-to-End Inference Configurations. We integrate

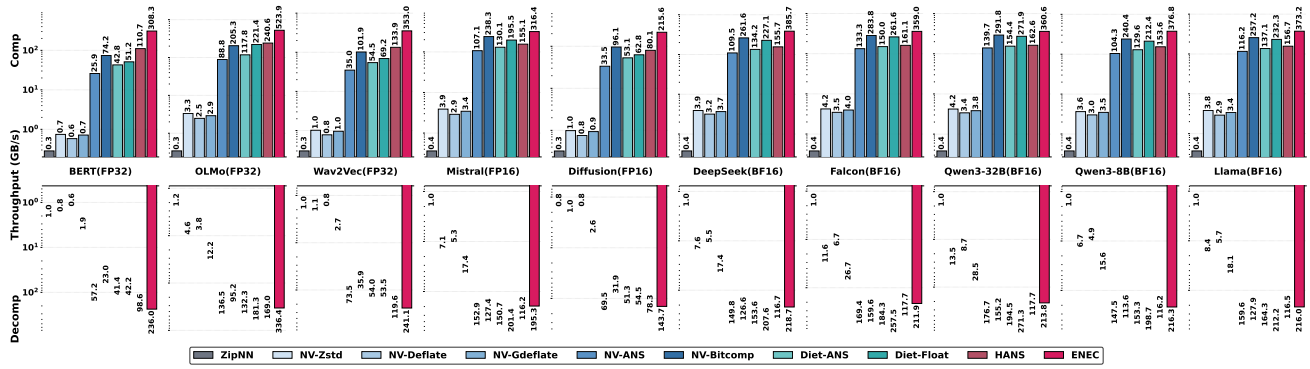

Fig. 9: Throughput of compression (upper) and decompression (lower) across different datasets and methods. The Y-axis (Throughput) is on a logarithmic scale to visualize performance differences spanning several orders of magnitude.

ENEC into the HuggingFace Transformers inference framework and evaluate it on two mainstream large language models, Qwen3-32B and Falcon-40B. The evaluation metrics are Time-To-First-Token (TTFT) and Time-Per-Output-Token (TPOT), measured using fixed input and output lengths. During inference, decompression is performed layer-wise. We overlap the next layer's decompression with the current layer's forward. The baseline uses an uncompressed inference setup with partial CPU offloading, since a single Ascend 910B2 cannot hold the full model weights.

A. Compression Ratio

The compression ratio results for all evaluated models are summarized in Table II. Importantly, we have meticulously verified that all 10 models (across FP32, FP16, and BF16 precisions) listed in Table III achieve **bit-identical reconstruction** via our compression and decompression pipeline.

For each model's weight data, we report the compression ratio by dividing the total size of all parameters before compression by the total size after compression. Since ZipNN, nvCOMP, DietGPU, and ENEC perform compression in file form, we first save the model's raw parameters as binary files before applying the respective file compression. For HANS, which only provides a Python API for compressing tensors, we load the model and then traverse each parameter tensor to conduct compression tests.

Table II shows ENEC's consistently strong performance across datasets. ENEC generally outperforms HANS and significantly exceeds nvCOMP's general-purpose methods. This advantage stems from our specialized exponent-mantissa separation. Unlike general compressors that process floating-point values uniformly—diluting exponent frequency statistics—our targeted linear mapping enables more accurate modeling and higher compression ratios.

ENEC remains competitive against SOTA specialized compressors. While ZipNN (CPU) and DietGPU::Float (GPU) achieve higher ratios, ENEC's performance is notably close on FP32 and FP16 datasets. This reflects a deliberate trade-off: ENEC prioritizes maximum throughput on resource-constrained Ascend NPUs via massive vectorization. While this architectural choice limits element-wise encoding flexibility—slightly reducing compression ratios—the substantial throughput gains meet high-performance data transmission demands. We argue this modest compromise is well justified by the significant practical efficiency.

B. Compression and Decompression Throughput

In this section, we evaluate and compare ENEC's compression and decompression throughput against other lossless compressors. Throughput is calculated by dividing the raw data size by the total compression or decompression time.

For ZipNN and DietGPU, we measure performance using their direct APIs. For nvCOMP, we use their benchmark code compiled with C++ and CUDA files, and report their direct presented results. For HANS and ENEC, we utilize the msprof profiling tool within the AscendC framework to assess kernel-level performance. This approach is adopted because HANS only provides a Python interface, and performance measurements at the Python level may differ significantly from the actual kernel-level performance. For both GPU and NPU platforms, reported throughput values measure device-side kernel execution time only, excluding PCIe transfer.

Compression Throughput. As shown in Figure 9, ENEC achieves significantly higher throughput across all data types compared to existing baseline platforms on CPUs, GPUs, and Ascend NPUs. Specifically, for BF16 models, ENEC achieves an average throughput of 372 GB/s, which is 987× higher than ZipNN—the SOTA compressor on CPUs—and 1.39× as much as nvCOMP:Bitcomp, the best-performing compressor on GPUs. It also outperforms HANS, another NPU-based compressor, by a factor of 1.36. For FP32 models, the performance gap remains consistent, with ENEC achieving throughput that is 1266× that of ZipNN, 3.08× that of Bitcomp, and 2.47× that of HANS. The performance for FP16 is similar to that of BF16, with ENEC achieving an average throughput of 263 GB/s, which is 767.33× faster than ZipNN, 0.58× higher than Bitcomp, and 2.27× that of HANS. These results strongly demonstrate the effectiveness of our lightweight and highly vectorized optimization approach tailored for Ascend NPUs' platforms.

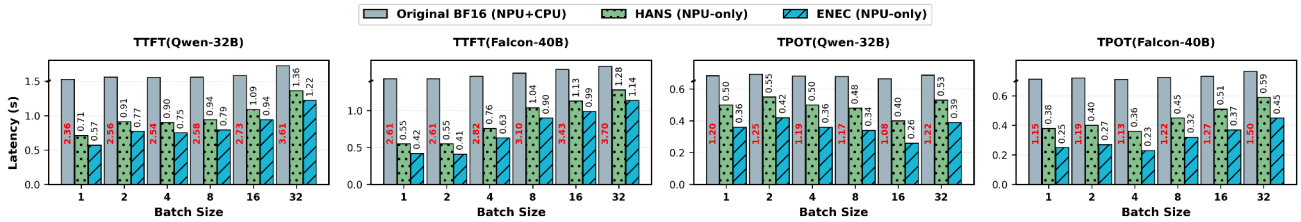

Fig. 10: End-to-end inference performance comparison of ENEC across different models and batch sizes. Latency metrics include Time-To-First-Token (TTFT) and Time-Per-Output-Token (TPOT).

Moreover, despite FP16's 5-bit exponent limiting compression and adding design challenges, our ENEC still performs nearly on par with BF16, reaching up to 317 GB/s, showcasing its efficiency across data types. Furthermore, ENEC achieves up to 523 GB/s on FP32—nearly twice BF16's speed. This is because we avoid compressing the mantissa. As data width doubles from 16-bit to 32-bit, the processed amount remains unchanged. Thus, compression time stays similar, significantly boosting throughput.

Decompression Throughput. Decompression throughput is measured and presented using the same methodology as compression throughput. Results across various model datasets are also shown in Figure 9. Compared to compression, both HANS and ENEC exhibit lower decompression throughput. This is primarily due to operations like prefix sum computation and reverse gather, which are not well optimized on Ascend NPU architectures, increasing computational overhead.

Nevertheless, ENEC maintains a comprehensive throughput advantage; its decompression performance significantly surpasses all other algorithms, achieving speedups of up to 4.22× over NV-Bitcomp and 2.11× over HANS, except for a 6% drop compared to Diet-Float on GPU for BF16 models. On FP16 and FP32 models, we still achieve an improvement of approximately 0.31–1.90 over Diet-Float. This gain stems from our optimizations in prefix sum computation and replacing lookup-table gather with linear mapping, which reduce memory access latency and enhance parallel efficiency.

TABLE IV: ENEC compression parameters adopted by the majority of tensors in the models.

BF16			FP16 & FP32		
Type	**Models**	**(b, n, m, L)**	**Type**	**Models**	**(b, n, m, L)**
BF16	Falcon	(122, 6, 3, 16)	FP16	Mistral	(7, 4, 3, 16)
BF16	Qwen3-8B	(123, 6, 3, 16)	FP16	Diffusion	(11, 5, 3, 16)
BF16	DeepSeek	(123, 6, 3, 16)	FP32	OLMo	(121, 6, 3, 16)
BF16	Qwen3-32B	(122, 6, 3, 16)	FP32	BERT	(123, 6, 3, 16)
BF16	Llama	(121, 6, 3, 16)	FP32	Wav2Vec	(125, 6, 3, 16)

C. End-to-End Inference Speedup

We evaluate the baseline and the ENEC-integrated inference framework on two mainstream models. Because a single NPU cannot host the full model, the baseline keeps most weights on the NPU and offloads only essential parts to the CPU. Our inference evaluation includes 10 warm-up and 50 test runs, from which we compute the average TTFT and TPOT. As shown in Figure 10, ENEC consistently surpasses both the uncompressed baseline and HANS. On Qwen3-32B, ENEC reduces TTFT by up to 4.1× and TPOT by up to 3.9× compared to the baseline; relative to HANS, ENEC achieves up to 1.7× lower TTFT and 1.6× lower TPOT. On Falcon-40B, ENEC attains maximum reductions of 6.3× in TTFT and 4.9× in TPOT over the baseline, and up to 1.8× and 1.7× improvements over HANS, respectively. These substantial improvements mainly result from eliminating the overhead of weight transfers and ENEC's superior decompression throughput. For instance, with a batch size of 4, transferring certain Falcon layer weights from CPU to NPU accounts for nearly 80% of execution time.

D. Evaluation of Parameter Tuning

This section presents a detailed experimental analysis of several parameters in our compressor, supplementing the theoretical analysis in Section V-E. We investigate the impact of data block size, group length (L), quantization parameters (n and m), and the integer transformation parameter (b).

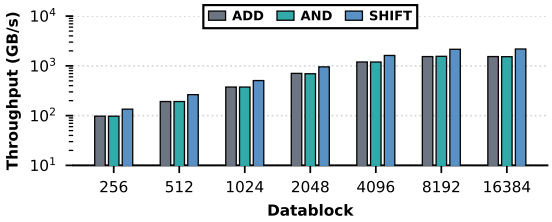

Fig. 11: Performance of several common operations under fixed input conditions across different data block sizes.

Data Block Size: This parameter primarily dictates system throughput with negligible compression impact. Throughput scales with block size (Fig. 11). For performance, we choose a 16,384-element block, avoiding 32,768 as its memory footprint exceeds UB's 192KB limit on Ascend NPUs.

Group Length (L): Our experimental results, summarized in Table IV, demonstrate that a group length of $L = 16$ consistently yields the optimal balance between granularity and compression efficiency. Notably, this value remains stable

across different data types (FP32, BF16, and FP16), suggesting that $L = 16$ provides an ideal alignment with the NPU's vectorized memory access patterns.

Quantization Parameters (b, n, m): These parameters are determined via the joint search optimization algorithm (Section V-E). We observe that for FP32 and BF16, the quantization parameters n and m remain highly consistent, with only minor variations in b due to data distribution differences. For FP16, despite its narrower 5-bit exponent, the optimal value for n remains 6, matching the other formats.

E. Parameter Robustness and Transferability

To assess ENEC robustness, we perform cross-model sensitivity analysis applying optimal parameters from DeepSeek-V3 to other LLMs (e.g., Qwen-3) without re-tuning.

TABLE V: Compression performance on other models using the parameters searched from DeepSeek-V3. "Optimal" is the performance achieved using the parameters searched on the target model.

Models	Compression Ratio	Comp. / Decomp. Thr. (GB/s)
Falcon-7B [46]	1.34 (Optimal: 1.34, 0%↓)	367 / 213 (Optimal: 359 / 212)
Qwen3-8B [50]	1.35 (Optimal: 1.35, 0%↓)	375.2 / 220.1 (Optimal: 377 / 216)
Qwen3-32B [50]	1.34 (Optimal: 1.34, 0%↓)	351 / 209 (Optimal: 361 / 214)
Llama-3.1-8B-Instruct [17]	1.31 (Optimal: 1.36, 5%↓)	338 / 199 (Optimal: 373 / 216)

Results in Table V show that the optimal parameters searched from DeepSeek-V3 transfer well across multiple models. For Falcon-7B, Qwen3-8B, and Qwen3-32B, compression remains fully lossless, and throughput even improves slightly, confirming ENEC stably delivers high performance under fixed structural parameters (group length L and bit-packing logic n, m). For Llama-3.1-8B-Instruct, a minor 5% loss in compression ratio and slight throughput decrease occur, but absolute performance remains relatively high. These fluctuations likely stem from differences in weight distribution across models, not algorithm design issues. Overall, the parameters searched by DeepSeek-V3 achieve zero-loss migration on most models, validating their effectiveness and robustness across different architectures.

F. Ablation Study

We perform an ablation study starting from baseline ENEC V0 (Section IV). V1 adds bit-width quantization with hierarchical halving bit-packing (Section V-B); V2 introduces vectorized branch-free integer transformation (Section V-C); and V3 incorporates *IDD-Scan* for decompression (Section V-D). We evaluate all versions and block size impact on DeepSeek [12] using ratio and throughput.

Analysis of Different Input File Sizes. Using DeepSeek as a case study, we analyzed compression across varying input sizes by partitioning parameters into different segment sizes and compressing them independently. As shown in Table VI and Figure 12, ENEC consistently outperforms baselines across all sizes, maintaining a stable advantage.

Analysis of Different ENEC Versions. Figure 13 compares compression ratio and throughput across versions. V0

TABLE VI: Compression ratios with different input file sizes (MB).

Arch	Compressors	1	2	4	8	16	32	64	128	256	512
CPU	ZipNN	1.51	1.51	1.51	1.51	1.51	1.51	1.51	1.50	1.50	1.50
GPU	NV_Zstd	1.28	1.28	1.28	1.28	1.28	1.28	1.28	1.27	1.27	1.28
	NV_Deflate	1.28	1.28	1.28	1.28	1.28	1.28	1.28	1.27	1.27	1.28
	NV_GDeflate	1.27	1.27	1.27	1.27	1.27	1.27	1.27	1.27	1.27	1.27
	NV_ANS	1.20	1.20	1.20	1.20	1.20	1.23	1.25	1.26	1.26	1.26
	NV_Bitcomp	1.33	1.33	1.33	1.33	1.33	1.33	1.33	1.36	1.36	1.36
	Diet_ANS	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
	Diet_Float	1.48	1.48	1.48	1.48	1.48	1.48	1.48	1.48	1.48	1.48
NPU	HANS	1.30	1.30	1.25	1.32	1.33	1.34	1.35	1.35	1.35	1.35
	ENEC	**1.37**	**1.37**	**1.37**	**1.37**	**1.38**	**1.37**	**1.38**	**1.37**	**1.37**	**1.38**

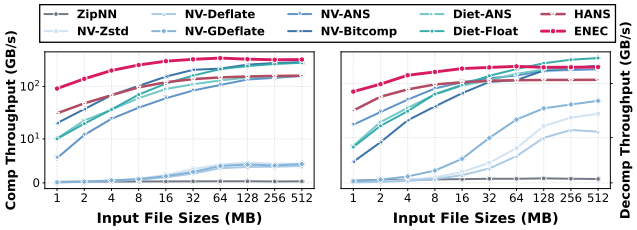

Fig. 12: Comparison of throughput performance across various methods under different input file sizes.

achieves a high ratio via frequency-based statistical mapping. V1 slightly improves on V0 by reducing per-group metadata from 4 bits to 1 bit. V2 and V3 adopt branchless integer transform; its linear mapping approximates the frequency distribution, slightly reducing the ratio. For throughput, V1 improves compression and decompression by nearly 30% over V0 using bit-width quantization with hierarchical bit-packing, replacing multiplication/division. V2 replaces slow gather with vectorized branch-free integer transformation, nearly doubling throughput on vector units. V3 further optimizes decoding with *IDD-Scan*, boosting decompression throughput by nearly 100% over V2.

VII. Discussion

This section discusses ENEC's generality and its adaptability to CPUs and GPUs, highlighting its modular design, which simplifies porting and optimization. We then derive architectural implications from our software–hardware co-design, offering insights for future accelerators to better support efficient lossless compression.

A. Generality of Proposed Design

Although ENEC's low-level implementation is deeply customized for the specific constraints of Ascend NPUs, its core design philosophy and algorithmic framework possess broad generality. The framework can be readily adapted to other accelerators, including NVIDIA GPUs.

Cross-Architecture Portability. The core components of ENEC—bit-width quantization, branch-free transformation, and hierarchical halving bit-packing—are designed to maximize efficiency by converting complex conditional logic into regularized, vectorized bitwise operations. These design principles are inherently portable across parallel computing architectures. To validate its portability, we implemented a baseline version (ENEC-GPU-V0) on the A800 GPU that

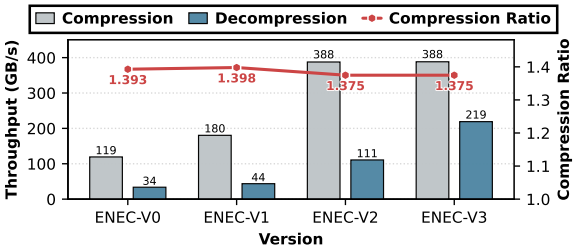

Fig. 13: Performance of several ENEC versions.

TABLE VII: Performance comparison of ENEC and its implementations on other platforms using Qwen-32B.

Hardware	Compressors	CR	Comp. / Decomp. Thr. (GB/s)
CPU	ZipNN	1.50	0.4 / 1.0
	ENEC-CPU	**1.35**	**1.22 / 1.56**
GPU	NV-ANS	1.24	139.7 / 176.7
	NV-Bitcomp	1.32	291.8 / 155.2
	Diet-ANS	1.23	154.4 / 194.5
	Diet-Float	1.47	271.9 / 271.3
	ENEC-GPU-V0	**1.34**	**291.3 / 269.4**
	ENEC-GPU-V1	**1.35**	**419.2 / 421.0**
NPU	HANS	1.34	162.6 / 117.7
	ENEC	**1.35**	**360.6 / 213.8**

strictly follows the original execution flow. As shown in Table VII, ENEC-GPU-V0 achieves a compression ratio of 1.34× and throughput of 291.3/269.4 GB/s, which is comparable to the NPU version's 1.35× ratio and 360.6/213.8 GB/s throughput. This demonstrates that ENEC's fundamental logic is independent of hardware instruction sets and can be seamlessly migrated across diverse architectures.

Hardware-Specific Optimizations. ENEC's modular design enables deep hardware-tailored tuning by substituting computational primitives. For instance, hierarchical halving bit-packing can be optimized using platform-specific shuffle or shift instructions. While the NPU version uses *IDD-Scan* to bypass alignment constraints, the GPU version (ENEC-GPU-V1) invokes optimized parallel prefix-sum from the CUB library, leverages warp-level intrinsics for fast communication, and utilizes vectorized memory access. These optimizations yield a 1.56× throughput improvement over the baseline (ENEC-GPU-V0). Similarly, our CPU implementation employs AVX2 for BF16 transformations and BMI2 PEXT for non-byte-aligned compression, achieving 1.56× the throughput of ZipNN in a single-core configuration.

B. Architectural Implications for Future Accelerators

The ENEC co-design highlights key principles for future architectures with compression acceleration:

- **Branch-free execution support:** Architectures should natively support branch-free execution via bit-manipulation and predication, aligning with prior compression studies [48].
- **Modular operator libraries:** Standardized interfaces for modular operator libraries enable vendor-optimized primitives (e.g., scans, gathers) while preserving portability.
- **Memory subsystem co-design:** Memory subsystems should be co-designed with data layouts to reduce the overhead of non-contiguous access.
- **Compression-specific hardware support:** We advocate for lightweight variable-length decoding near vector registers, dedicated per-lane bit-extraction, and fast parallel prefix-sum units to replace multi-stage workarounds such as *IDD-Scan*.
- **Low-precision datapath specialization:** Specialized datapaths for low-precision formats should decouple exponent and mantissa processing and support hardware-level adaptive bit-width packing.

While some insights echo prior work, this study is the first to articulate them in the context of accelerator design.

VIII. Related Work

The most relevant works to ENEC are DietGPU [31], an efficient lossless compressor for GPUs, and HANS [3], which is designed for Ascend NPUs. DietGPU, developed by Facebook Research, provides fast and specialized lossless compression on NVIDIA GPUs for both machine learning and HPC workloads. It includes a GPU-optimized ANS entropy coder to achieve high throughput and exposes two interfaces: a general "ANS" mode and a specialized "Float" mode tailored for model data. The "Float" mode compresses floating-point values by isolating and encoding only their exponents, a common strategy in weight compression. HANS [3] is an efficient closed-source lossless compressor developed by Huawei on Ascend NPUs.

IX. Conclusion and Future Work

This paper presents a lossless compressor tailored for Ascend NPUs, specifically for model weight compression. Experiments show that ENEC achieves compression throughput of 263–523 GB/s and decompression throughput of 188–336 GB/s on FP32, FP16, and BF16 model weights. On 910B2 NPU, it delivers 2.47× higher compression throughput and 2.11× faster decompression throughput than SOTA methods, while also achieving superior compression ratios. It further demonstrates excellent performance in end-to-end inference scenarios. Efficient compression is achieved through hardware-informed decomposition and data-dependent pipelining, balancing efficiency and compression ratio via an "approximate majority + precise correction" paradigm with trade-offs centered on system bottlenecks. As hardware and architecture technology evolves, we plan to extend support and performance optimizations to a broader range of low-precision and integer data formats.

Acknowledgments

This work was supported by the Innovation Funding of ICT, CAS (Grant No. E461050), the National Key Research and Development Program of China (Grant No. 2025YFB3003702), and the National Natural Science Foundation of China (Grant Nos. 62032023 and T2125013). The experiments were performed on the robotic AI-Scientist platform of Chinese Academy of Sciences.

REFERENCES

[1] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann *et al.*, "Phi-4 technical report," *arXiv preprint arXiv:2412.08905*, 2024.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] H. Ascend, "Hans documentation," https://gitee.com/ascend/op-plugin/pulls/2449/files, 2025.

[4] S. Ashkboos, M. L. Croci, M. G. d. Nascimento, T. Hoefler, and J. Hensman, "Slicegpt: Compress large language models by deleting rows and columns," *arXiv preprint arXiv:2401.15024*, 2024.

[5] N. Azami, A. Fallin, and M. Burtscher, "Efficient lossless compression of scientific floating-point data on cpus and gpus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 395–409.

[6] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts *et al.*, "Stable video diffusion: Scaling latent video diffusion models to large datasets," *arXiv preprint arXiv:2311.15127*, 2023.

[7] burtscher, "LC-framework," https://github.com/burtscher/LC-framework, 2025.

[8] Cerebras, "Cerebras Training and Inference Docs," https://docs.cerebras.net/en/latest, 2024.

[9] X. Chen, J. Tian, I. Beaver, C. Freeman, Y. Yan, J. Wang, and D. Tao, "Fcbench: Cross-domain benchmarking of lossless compression for floating-point data," *arXiv preprint arXiv:2312.10301*, 2023.

[10] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," Tech. Rep., 2018.

[11] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," *arXiv preprint arXiv:2006.13979*, 2020.

[12] DeepSeek-AI, "Deepseek llm: Scaling open-source language models with longtermism," *arXiv preprint arXiv:2401.02954*, 2024. [Online]. Available: https://github.com/deepseek-ai/DeepSeek-LLM

[13] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," 2022. [Online]. Available: https://arxiv.org/abs/2208.07339

[14] H. Developers, "Document," https://developer.huawei.com/consumer/en/doc/hiai-guides/introduction-0000001051486804, 2025.

[15] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," *CoRR*, vol. abs/1810.04805, 2018. [Online]. Available: http://arxiv.org/abs/1810.04805

[16] B. Du and Z. Ye, "A novel method of lossless compression for 2-d astronomical spectra images," *Experimental Astronomy*, vol. 27, no. 1, p. 19, 2009.

[17] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv e-prints*, pp. arXiv–2407, 2024.

[18] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of huffman coding with compression rate of arithmetic coding," *arXiv preprint arXiv:1311.2540*, 2013.

[19] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," in *International conference on machine learning*. PMLR, 2023, pp. 10 323–10 337.

[20] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[21] D. Groeneveld, I. Beltagy, P. Walsh, A. Bhagia, R. Kinney, O. Tafjord, A. H. Jha, H. Ivison, I. Magnusson, Y. Wang, S. Arora, D. Atkinson, R. Authur, K. Chandu, A. Cohan, J. Dumas, Y. Elazar, Y. Gu, J. Hessel, T. Khot, W. Merrill, J. Morrison, N. Muennighoff, A. Naik, C. Nam, M. E. Peters, V. Pyatkin, A. Ravichander, D. Schwenk, S. Shah, W. Smith, E. Strubell, N. Subramani, M. Wortsman, P. Dasigi, N. Lambert, K. Richardson, L. Zettlemoyer, J. Dodge, K. Lo, L. Soldaini, N. A. Smith, and H. Hajishirzi, "Olmo: Accelerating the science of language models," *Preprint*, 2024.

[22] Groq, "GroqFlow provides an automated tool flow for compiling machine learning and linear algebra workloads into Groq programs and executing those programs on GroqChip™ processors." https://github.com/groq/groqflow, 2025.

[23] A. Heilper and D. Singer, "Lossless compression of neural network components: Weights, checkpoints, and k/v caches in low-precision formats," *arXiv preprint arXiv:2508.19263*, 2025.

[24] M. Hershcovitch, A. Wood, L. Choshen, G. Girmonsky, R. Leibovitz, O. Ozeri, I. Ennmouri, M. Malka, P. Chin, S. Sundararaman *et al.*, "Zipnn: Lossless compression for ai models," in *2025 IEEE 18th International Conference on Cloud Computing (CLOUD)*. IEEE, 2025, pp. 186–198.

[25] T. Hidayat, M. H. Zakaria, and A. N. C. Pee, "Increasing the huffman generation code algorithm to equalize compression ratio and time in lossless 16-bit data archiving," *Multimedia tools and applications*, vol. 82, no. 16, pp. 24 031–24 068, 2023.

[26] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 2007.

[27] K. V. Iyer, K. Seshadri, and K. Srinivasulu, "An advancement in huffman coding with a potential for parallel decoding," *Concurrency and Computation: Practice and Experience*, vol. 37, no. 9-11, p. e70096, 2025.

[28] J. Jia, C. Xie, H. Lu, D. Wang, H. Feng, C. Zhang, B. Sun, H. Lin, Z. Zhang, X. Liu *et al.*, "Sdp4bit: Toward 4-bit communication quantization in sharded data parallelism for llm training," *Advances in Neural Information Processing Systems*, vol. 37, pp. 8734–8759, 2024.

[29] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," *ArXiv*, vol. abs/2310.06825, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:263830494

[30] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong *et al.*, "{MegaScale}: Scaling large language model training to more than 10,000 {GPUs}," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 745–760.

[31] J. Johnson, "GPU implementation of a fast generalized ANS (asymmetric numeral system) entropy encoder and decoder," https://github.com/facebookresearch/dietgpu, 2025.

[32] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen *et al.*, "A study of bfloat16 for deep learning training," *arXiv preprint arXiv:1905.12322*, 2019.

[33] F. Knorr, P. Thoman, and T. Fahringer, "ndzip-gpu: efficient lossless compression of scientific floating-point data on gpus," in *Proceedings of the international conference for high performance computing, networking, storage and analysis*, 2021, pp. 1–14.

[34] G. G. Langdon, "An introduction to arithmetic coding," *IBM Journal of Research and Development*, vol. 28, no. 2, pp. 135–149, 1984.

[35] J. Lee, J. Bae, B. Kim, S. J. Kwon, and D. Lee, "To fp8 and back again: Quantifying reduced precision effects on llm training stability," 2025. [Online]. Available: https://arxiv.org/abs/2405.18710

[36] R. Li, Z. Li, Y. Wu, C. Chen, and Y. Zheng, "Elf: Erasing-based lossless floating-point compression," *Proceedings of the VLDB Endowment*, vol. 16, no. 7, pp. 1763–1776, 2023.

[37] S. Li, K. Lu, Z. Lai, W. Liu, K. Ge, and D. Li, "A multidimensional communication scheduling method for hybrid parallel dnn training," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 8, pp. 1415–1428, 2024.

[38] Z. Li, R. Li, X. Xu, Y. Wu, C. Chen, T. Liu, J. Shang, and Y. Zheng, "Adaptive encoding strategies for lossless floating-point compression," *IEEE Internet of Things Journal*, 2025.

[39] H. Liao, J. Tu, J. Xia, and X. Zhou, "Davinci: A scalable architecture for neural network computing," in *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, 2019, pp. 1–44.

[40] F. Lin, K. Arunruangsirilert, H. Sun, and J. Katto, "Recoil: Parallel rans decoding with decoder-adaptive scalability," in *Proceedings of*

the 52nd International Conference on Parallel Processing, 2023, pp. 31–40.

[41] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.

[42] Z. Liu, S. Cheng, G. Tan, Y. You, and D. Tao, "Elasticmm: Efficient multimodal llms serving with elastic multimodal parallelism," *arXiv preprint arXiv:2507.10069*, 2025.

[43] T. Lu, Y. Zhong, Z. Sun, X. Chen, Y. Zhou, F. Wu, Y. Yang, Y. Huang, and Y. Yang, "Adt-fse: A new encoder for sz," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–13.

[44] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, "Fp8 formats for deep learning," 2022. [Online]. Available: https://arxiv.org/abs/2209.05433

[45] NVIDIA, "Nvidia nvcomp developer," https://developer.nvidia.com/nvcomp, 2025.

[46] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay, "The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only," *arXiv preprint arXiv:2306.01116*, 2023. [Online]. Available: https://arxiv.org/abs/2306.01116

[47] J. Pieprzyk, J. Duda, M. Pawłowski, S. Camtepe, A. Mahboubi, and P. Morawiecki, "The compression optimality of asymmetric numeral systems," *Entropy*, vol. 25, no. 4, p. 672, 2023.

[48] M. Shah, X. Yu, S. Di, M. Becchi, and F. Cappello, "Lightweight huffman coding for efficient gpu compression," in *Proceedings of the 37th International Conference on Supercomputing*, 2023, pp. 99–110.

[49] S. Systems, "SambaNova :: SambaNova Documentation," https://docs.sambanova.ai/home/latest/index.html, 2024.

[50] Q. Team, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[51] X. Wang, Y. Zheng, Z. Wan, and M. Zhang, "Svd-llm: Truncation-aware singular value decomposition for large language model compression," *arXiv preprint arXiv:2403.07378*, 2024.

[52] A. Weißenberger and B. Schmidt, "Massively parallel ans decoding on gpus," in *Proceedings of the 48th International Conference on Parallel Processing*, 2019, pp. 1–10.

[53] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 06, pp. 8–19, 1984.

[54] H. Xi, H. Cai, L. Zhu, Y. Lu, K. Keutzer, J. Chen, and S. Han, "Coat: Compressing optimizer states and activation for memory-efficient fp8 training," 2025. [Online]. Available: https://arxiv.org/abs/2410.19313

[55] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.

[56] H. Yamamoto and K.-i. Iwata, "Asymptotic optimality of the asymmetric encoding-decoding scheme," in *2024 International Symposium on Information Theory and Its Applications (ISITA)*. IEEE, 2024, pp. 354–359.

[57] N. Yamamoto, K. Nakano, Y. Ito, D. Takafuji, A. Kasagi, and T. Tabaru, "Huffman coding with gap arrays for gpu acceleration," in *Proceedings of the 49th International Conference on Parallel Processing*, 2020, pp. 1–11.

[58] Z. Yuan, Y. Shang, Y. Song, Q. Wu, Y. Yan, and G. Sun, "Asvd: Activation-aware singular value decomposition for compressing large language models," *arXiv preprint arXiv:2312.05821*, 2023.

[59] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee *et al.*, "Llm inference unveiled: Survey and roofline model insights," *arXiv preprint arXiv:2402.16363*, 2024.

[60] P. Yubeaton, T. Mahmoud, S. Naga, P. Taheri, T. Xia, A. George, Y. Khalil, S. Q. Zhang, S. Joshi, C. Hegde *et al.*, "Huff-llm: End-to-end lossless compression for efficient llm inference," *arXiv preprint arXiv:2502.00922*, 2025.

[61] B. Zhang, J. Tian, S. Di, X. Yu, M. Swany, D. Tao, and F. Cappello, "Gpulz: Optimizing lzss lossless compression for multi-byte data on modern gpus," in *Proceedings of the 37th International Conference on Supercomputing*, 2023, pp. 348–359.

[62] T. Zhang, Y. Sui, S. Zhong, V. Chaudhary, X. Hu, and A. Shrivastava, "70% size, 100% accuracy: Lossless llm compression for efficient gpu inference via dynamic-length float," *arXiv preprint arXiv:2504.11651*, 2025.

[63] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li *et al.*, "A survey on efficient inference for large language models," *arXiv preprint arXiv:2404.14294*, 2024.

[64] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 2003.

[65] ——, "A universal algorithm for sequential data compression," *IEEE Transactions on information theory*, vol. 23, no. 3, pp. 337–343, 2003.

[66] P. Zuo, H. Lin, J. Deng, N. Zou, X. Yang, Y. Diao, W. Gao, K. Xu, Z. Chen, S. Lu *et al.*, "Serving large language models on huawei cloudmatrix384," *arXiv preprint arXiv:2506.12708*, 2025.

APPENDIX

A. Abstract

The source code of ENEC is available at https://github.com/jinwuyang/ENEC_ISCA_AE. The NPU kernel implementation can be found in the **csrc/** directory and the Python-based data processing, parameter search, and profiling scripts can be found in the **python/** directory. Since this paper includes a large number of experiments that in aggregate will take 24 hours to fully test all model compression tasks, the instructions here will focus on reproducing the results for **Qwen3-32B**. The workflow to reproduce other models is very similar.

B. Artifact check-list (meta-information)

- **Algorithm:** ENEC (Efficient NPU-Enhanced Compression).
- **Program:** Python 3.9, C++/Ascend C.
- **Compilation:** CMake 3.10+, GCC 7.5+ (aarch64), Ascend CANN Toolkit.
- **Transformations:** Layer-wise weight splitting.
- **Binary:** Compiled NPU operators (.so files) and executables.
- **Data set:** Qwen3-32B.
- **Run-time environment:** Ubuntu22.04, CANN8.2.RC1.alpha002.
- **Hardware:** Ascend 910B2 NPU.
- **Run-time state:** Isolated NPU execution.
- **Execution:** Automated profiling via msprof.
- **Metrics:** Compression Ratio (CR), Throughput (GB/s).
- **Output:** CSV reports and summary files.
- **Experiments:** Data preparation, parameter search, performance benchmarking, and inference.
- **How much disk space required (approximately)?:** 200 GB.
- **How much time is needed to prepare workflow (approximately)?:** 1 hours.
- **How much time is needed to complete experiments (approximately)?:** 3 hours.
- **Publicly available?:** Yes.
- **Code licenses:** BSD-3.

C. Description

1) How to access: The source code is available at https://github.com/jinwuyang/ENEC_ISCA_AE. The repository is organized into **csrc/** (NPU kernels), **python/** (test tools).

2) Hardware dependencies: The artifact requires an **Ascend 910B2 NPU** platform with **aarch64** architecture.

3) Software dependencies:

- **CANN Stack:** Ascend-CANN-toolkit and Kernels 8.2.RC1.alpha002.
- **Python Libraries:** torch 2.5.1, torch_npu 2.5.1.post3, and standard data science stack (numpy, pandas, scipy).
- **ATB Library:** Recommended version 8.0.0.

4) Data sets: The evaluation of ENEC encompasses a diverse set of model weights, categorized by their data precision formats. By default, the data_prepare.sh script only downloads **Qwen3-32B** to minimize preparation time and disk usage. However, the data_prepare.sh script provides commented options to download all other models listed below (e.g., DeepSeek-LLM-7B-Base, Falcon-7B, etc.) for users who wish to reproduce the complete set of experiments.

BF16 Models:

```
DeepSeek-LLM-7B-Base
Falcon-7B
Qwen3-8B
Llama-3.1-8B-Instruct
Qwen3-32B
Falcon-40B
```

FP16 Models:

```
CapybaraHermes-2.5-Mistral-7B
stable-video-diffusion-img2vid-fp16
```

FP32 Models:

```
OLMo-1B-hf
bert-base-uncased
wav2vec2-large-xlsr-53-english
```

D. Installation

1) Install CANN Toolkit and Kernels: Download the following files from https://www.hiascend.com/developer/download/community/result?module=cann&cann=8.2.RC1.alpha002:

- Ascend-cann-toolkit_8.2.RC1.alpha002_linux-aarch64.run
- Ascend-cann-kernels-910b_8.2.RC1.alpha002_linux-aarch64.run

Then run the following commands:

```
# Add executable permissions
chmod +x Ascend-cann-toolkit_8.2.RC1.alpha002_linux-\
    aarch64.run
chmod +x Ascend-cann-kernels-910b_8.2.RC1.alpha002_linux-\
    aarch64.run
# Verify the installers
./Ascend-cann-toolkit_8.2.RC1.alpha002_linux-aarch64.run --\
    check
./Ascend-cann-kernels-910b_8.2.RC1.alpha002_linux-aarch64.\
    run --\check
# Install
./Ascend-cann-toolkit_8.2.RC1.alpha002_linux-aarch64.run --\
    install --install-path=/your/path
./Ascend-cann-kernels-910b_8.2.RC1.alpha002_linux-aarch64.\
    run --install --install-path=/your/path
source /your/path/ascend-toolkit/set_env.sh
```

2) Configure the Conda environment: Create a Python 3.9 environment and install NPU-specific PyTorch and dependencies:

```
conda create -n enec python=3.9 -y
conda activate enec
pip install pandas numpy==1.24.3 transformers==4.30.0 jinja2 \
    decorator attrs psutil absl-py cloudpickle ml-dtypes scipy \
    tornado pyyaml
wget https://download.pytorch.org/whl/cpu/torch-2.5.1-cp39-\
    cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.\
    whl
pip install torch-2.5.1-cp39-cp39-manylinux_2_17_aarch64.\
    manylinux2014_aarch64.whl
wget https://gitee.com/ascend/pytorch/releases/download/v7
    .1.0.2-pytorch2.5.1/torch_npu-2.5.1.post3-cp39-cp39-\
    manylinux_2_17_aarch64.manylinux2014_aarch64.whl
pip install torch_npu-2.5.1.post3-cp39-cp39-manylinux_2_17_\
    aarch64.manylinux2014_aarch64.whl
```

3) Verify the environment: Run a simple NPU tensor operation to confirm correct setup:

```
python3 -c "import torch; import torch_npu; a = torch.randn(3,
    4).npu(); print(a + a)"
```

If the output is normal, the environment is normal.

4) Build: Clone the repository and run build_csrc.sh (1 hour).

```
git clone https://github.com/jinwuyang/ENEC_ISCA_AE.git
chmod 777 -R ENEC_ISCA_AE
cd ENEC_ISCA_AE
bash build_csrc.sh
```

E. Experiment workflow

1) Data Preparation: Execute data_prepare.sh to download and split the model weights. By default, the script only downloads and processes **Qwen3-32B** (1 hour). To test other models (e.g., DeepSeek-LLM-7B, Falcon-40B), simply uncomment the corresponding lines in data_prepare.sh.

```
bash data_prepare.sh
```

2) Performance Testing: Run compressor_test.sh to measure the compression ratio and throughput. This script automates parameter searching, compression/decompression profiling, and global analysis. At the end of the execution, it also outputs the end-to-end inference results (2 hours).

```
source /your/path/ascend-toolkit/set_env.sh
bash compressor_test.sh
```

F. Evaluation and expected results

1) Optimal parameter search results: The following results show the expected outputs for the Qwen3-32B model:

```
                BF16 Model Compression Results
========================================
File Processed:       hyperparams_results.csv
Total Elements:       32,761,446,400
------------------------------------------------
Original BF16 Size:       62487.50 MB
ENEC Compressed Size:     46265.99 MB
Compression Ratio (CR):       1.35x
Model Avg Bit-width:      11.8465 bits/element
Exponent Avg Bit-width:    3.8465 bits/element
Formula Avg CR*:              1.35 x
```

The optimal parameter search results are organized within the **param_search_enec/** directory. Each model subfolder (e.g., BF16/Qwen3-32B) provides:

- **hyperparams_results.csv**: An exhaustive list of optimal parameters for every model tensor.
- **param_combinations_stats.txt**: A comprehensive statistical report of the search results.

2) Compression Ratio and Throughput: The file summary_enec.csv summarizes the compression ratio, compression throughput, and decompression throughput of ENEC on **11 models**, corresponding to Table II and Figure 9 in the paper. The expected results for these 11 models are presented as follows:

```
--- Summary Data Preview ---
model_name dtype  compression_ratio_CR
    compress_throughput_GBps
    decompress_throughput_GBps
Llama-3.1-8B-Instruct  BF16  1.36
    376.8  219.4
Qwen3-32B  BF16  1.35  366.3
    217.1
Qwen3-8B  BF16  1.36  388.1
    222.7
deepseek-llm-7b-base  BF16  1.37
    391.2  223.1
falcon-40b  BF16  1.37  369.1
    217.2
falcon-7b  BF16  1.36  364.6
    215.8
CapybaraHermes-2.5-Mistral-7B  FP16  1.12
    317.0  195.5
stable-video-diffusion-img2vid  FP16  1.09
    223.4  148.0
OLMo-1B-hf  FP32  1.15  538.6
    348.7
bert-base-uncased  FP32  1.15
    329.1  252.8
wav2vec2-large-xlsr-53-english  FP32  1.15
    372.1  254.6
```

3) End-to-End Inference Latency: Figure 10 in the paper shows the end-to-end inference latency and speedup over the baseline (uncompressed with CPU offloading) for both Qwen3-32B and Falcon-40B under different batch sizes. For brevity, we only present the results for **Qwen3-32B** with **batch size = 1**. The expected results are presented as follows:

```
[Inference: Qwen3-32B]
  Configuration: size=61.02 GB, throughput=217.05 GB/s
  baseline TTFT: 2.36064 s
  baseline TPOT: 1.1951 s
  ENEC TTFT: 0.556342 s (Speedup: 4.24x)
  ENEC TPOT: 0.361142 s (Speedup: 3.31x)
  Result saved to: Latency_Qwen3-32B_BF16.csv
```